

Einführung in den systematischen Softwaretest

Modul II2005 im BSc Informatik der THM

Andreas Gareiß

Fachbereich Mathematik, Naturwissenschaften und Informatik

29. Oktober 2025



CAMPUS
GIESSEN

MNI

Mathematik, Naturwissenschaften
und Informatik

Einige wichtige Begriffe beim Softwaretest

Was ist ein Softwarefehler?

- Nicht anforderungskonformes Verhalten des Programms
- Nichterfüllung einer zuvor festgelegten Anforderung
- Abweichung zwischen Spezifikation (Sollverhalten) und tatsächlichem Verhalten (Istverhalten)
- Softwarefehler können die Funktionalität beeinträchtigen
- Softwarefehler werden im Betrieb oder (besser) vorher beim Test festgestellt
- Andere Bezeichnungen: Failure, Fehlerwirkung, Fehlfunktion, Ausfall, äußerer Fehler

Wie entsteht ein Softwarefehler?

- Softwarefehler entstehen durch *Defekte* im Programm
- Defekte sind Irrtümer in der Programmierung, z.B. falsche Anweisungen, vergessene Anweisungen fehlerhafte Kontrollstrukturen
- Defekte können lange unentdeckt bleiben
- Defekte führen nicht zum Softwarefehler, wenn dieser Zweig nicht ausgeführt wird
- Defekte heißen auch fault, Fehlerzustand oder innerer Fehler

Was ist ein Softwaretest?

Ein Softwaretest

- vergleicht das Verhalten einer SW-Komponente mit der Spezifikation bzw. dem gewünschten Verhalten
- vergleicht also Istverhalten mit Sollverhalten
- hat das Ziel Softwarefehler (äußere Fehler) zu entdecken und zu beschreiben, wann sie auftreten
- ergibt eine Aussage zu Entwicklungszustand und Qualität der Software
- kann das Vertrauen in die Software erhöhen
- kann Schwächen in der Spezifikation aufdecken

Unterschied Softwaretest vs. Debuggen

- Softwaretest** Verfahren um äußere Fehler aufzudecken; orientiert sich überwiegend am Ergebnis und der Wirkung von SW-Komponenten. Bei manchen Verfahren ist die Struktur der SW völlig unbekannt (Blackbox-Test) bei anderen wird sie benutzt (Whitebox-Test, Reviews)
- Debugging** Das gezielte Durchgehen des Programms um den inneren Fehler (Defekt) zu finden, der einen äußeren Fehler verursacht. Dabei werden die typischen Hilfsmittel des Debuggens benutzt: Breakpoints, Einzelschrittausführung, Watches, View auf Speicher und Register

Testbegriffe

Testprozess Testobjekt wird mit Testdaten ausgeführt

Testmanagement Planung, Durchführung und Auswertung

Testlauf Ausführung von 1..n Testfällen

Testfall Besteht aus Randbedingung(en), Eingabewerte(en), Ausgabewert(en) bzw. Verhalten. Er sollte eine gewissen Wahrscheinlichkeit besitzen, dass er auch eintritt.

Testscenario 1..n Testfälle hintereinander wobei die Ausgabe des vorherigen die Eingabe des nächsten Testfalls ist

Testbegriffe

Testziel Allgemein: Das Aufdecken von Fehlern.
Präzise: Ziele z.B. **bestimmte** Reaktion,
Geschwindigkeit, Korrektheit, ...

Testmethode Regressionstest,
Internationalisierungstest, Lasttest,
Review, ...

Testobjekt Beispiel hierfür sind GUI oder DB

Teststufe Bezieht sich auf das V-Modell.
Komponententest, Integrationstest,
Systemtest oder Abnahmetest

Testperson Durchführende Person / Funktion im
Projekt. Entwickler, Tester oder auch
Anwender

V-Modell

Das V-Modell ist ein Vorgehensmodell für die Softwareerstellung und -validierung. Oben ist die Ebene der Spezifikation, unten die Ebene mit allen Details (Code).

Linker Zweig des V-Modells Es geht von der Spezifikation über Grobentwurf und Feinentwurf bis herab zur Kodierung.

Rechter Zweig des V-Modells Zu jeder Ebene gibt es ein Validierungsverfahren; dabei bewegt man sich wieder von der Kodierungsebene, wo debuggt wird über den Unit-Test und den Integrationstest aufwärts bis zum Test des ganzen Systems. Dieser hat verschiedene Ausprägungen: Stresstest, Akzeptanztest.

V-Modell

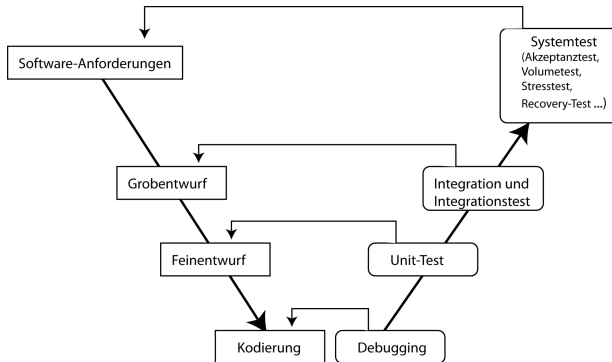


Abbildung: V-Modell

Komponententest

Alle Tests, bei denen Bestandteile der Software getestet werden und nicht die Software als Ganzes. Auch: Unit-Test, Modul-Test, Klassen-Test.

- Statische Tests
- Dynamische Tests

(Dazu später mehr)

Integrationstest

Integration ist das Zusammenfügen der Komponenten. Hier treten häufig Inkompatibilitäten und andere Probleme auf. Für den Integrationstest werden kleinere Komponenten zu größeren Einheiten zusammengefügt und die Software in größeren Einheiten oder als Ganzes getestet. Das Zusammenfügen kann schrittweise (inkrementell, verschiedene Strategien) oder komplett erfolgen (Big Bang). Der Integrationstest schließt sich dem Unit-Test an und setzt voraus, dass die Units getestet sind.

Systemtest

Die 3. Teststufe, sollte nach dem Integrationstest stattfinden. Dabei werden alle Komponenten zusammengefügt und die Software als Ganzes getestet. Dabei wird nun mehr die Sicht des Anwenders berücksichtigt und weniger gegen rein technische Spezifikationen getestet. Viel Funktionen und Features können jetzt erst getestet werden, wenn alle Komponenten zusammengefügt sind. Deshalb wird hier auch eine neue Testumgebung gebraucht, z.B. mit Kundendaten.

Abnahmetest

Hier steht nun die Sicht und Spezifikation des Kunden ganz im Vordergrund. Der Abnahmetest hat auch rechtliche Konsequenzen, z.B. dass das Betriebsrisiko nun auf den Kunden übergeht. Formen des Abnahmetests sind:

- Test auf vertragliche Akzeptanz
- Test der Benutzerakzeptanz
- Akzeptanz durch Systembetreiber (z.B. bei App)
- Feldtest (Alpha- und Betatest)

Funktionale Tests

Hier steht die Erbringung der gewünschten Funktionen im Vordergrund, sie werden sehr genau und detailliert gegen die Anforderungen geprüft. (Komponententests, Integrationstest, Systemtest)

Nichtfunktionale Tests

Lasttest Systemverhalten bei steigender Systemlast

Performanztest Messung der Verarbeitungsgeschwindigkeit
bzw. der Antwortzeiten

Volumentest Systemverhalten bei steigender Datenmenge

Stresstest Verhalten bei Überlastung

Test der Datensicherheit Abschirmung gegen unberechtigten
Systemzugang

Stabilitätstest Ausfallrate im Dauerbetrieb

Test auf Robustheit Fehlbedienung, Wiederanlaufverhalten

Kompatibilitätstest Verträglichkeit mit vorhandenen Systemen,
Import, Export

Statische Tests

Alle Tests, bei denen der Code **nicht** zur Ausführung gebracht wird. Ausprägungen sind:

Strukturierte Gruppenprüfungen Dazu zählen alle Prüfungen, bei denen der Code von erfahrenen Entwicklern durchgesehen und diskutiert wird: Review, Walkthroughs und Inspektionen

Statische Analyse Hier werden Werkzeuge (Softwaretools) benutzt, die den Code automatisch analysieren und entsprechende Protokolle mit Ergebnissen auswerfen.

Dynamische Tests

Alle Tests, bei dem der Code zur Ausführung gebracht wird. Für diese Ausführung muss eine Laufzeitumgebung (Testbett) erstellt werden und es müssen Eingabedaten (Testfälle) entworfen werden. Ausprägungen sind:

Blackbox-Test Die Struktur der SW-Komponente bleibt unbekannt (Blackbox) und es wird gegen die Spezifikation getestet. Beim Entwurf der Testfälle orientiert man sich an der Spezifikation.

Whitebox-Test Die Struktur des Programmes liegt offen und dient als Grundlage für den Entwurf der Testfälle.

Mehr lesen ...

Zu den Grundbegriffen des Testens findet man natürlich bei fast allen Quellen etwas. Eine sehr schöne Zusammenfassung auf ca. 30 Seiten findet man bei Krümmel und Iglar [Nad11]. Ein bewährtes Werk ist das Buch von Myers [Mye01]. Mit einem ausführlichen Beispiel (Kalenderroutinen) arbeitet Thaller [Tha02]; sehr systematisch sind Spillner und Linz [And19].

Wichtige Begriffe beim Softwaretest

- Softwarefehler
- Softwaretest
- V-Modell
- Testausprägungen
- Testarten
- Statische und dynamische Tests
- Literatur

Literatur